

# Project 1: Motion Detection

Charles Ciampa  
Khoury College of Computer Science  
Northeastern University  
Boston, Massachusetts  
ciampa.c@northeastern.edu

Abhishek Narang  
College of Engineering  
Northeastern University  
Boston, Massachusetts  
narang.ab@northeastern.edu

**Abstract**—Motion detection in video sequences captured by stationary cameras is a fundamental concept in computer vision with applications spanning surveillance, traffic monitoring, and activity recognition. This experiment explores a temporal filtering approach to detecting moving objects through analyzing intensity changes at each particular pixel over time. It computes temporal derivatives using both simple filters and Gaussian derivative filters with varying standard deviations. To reduce noise, spatial preprocessing techniques were investigated such as box filters and 2D Gaussian smoothing before temporal differentiation. An adaptive thresholding strategy was tested that estimates background noise and applies a  $\sigma$  threshold to distinguish motion from stationary components. Experimental results on different video sequences demonstrate that a 5 by 5 Gaussian spatial smoothing with a standard deviation of 1 and a Gaussian temporal derivatives of 1.0 and adaptive thresholding with a value 5 times the standard deviation leads to optimal performance, successfully detecting motion while minimizing chance of false positives. This approach has robust motion detection across varying scenes without needing training data or much manual tuning.

## I. INTRODUCTION

Motion detection is a fundamental concept in computer vision applications, ranging from surveillance to tracking to autonomous vehicles. As technologies and algorithms in this space have been further developed, more challenges have been identified. One of these that lies at the core is distinguishing genuine motion from noise found in stationary camera footage. This is common because not every camera used for sourcing footage for motion detection is perfectly ideal, with no noise or devoid of other limitations. Most pixels however, belong to the stationary background, with moving objects causing very noticeable, rapid intensity changes.

The problem statement here gives us a sequence of 300-1100 frames from a stationary camera set in a location with multiple instances of different objects moving. Through analyzing the temporal evolution of pixel intensities, the objective of detecting and localizing moving objects in each frame is achieved. The intensity values observed at a particular pixel over time is constant or slowly varying, except for when a moving object passes through that pixel. This would cause a big change in the pixel intensity compared with time. Therefore, by looking at the rate of change of intensity with respect to time, if a large temporal gradient is detected, motion is likely.

## II. DESCRIPTION OF ALGORITHMS

This project employed the use of 3 separate methods of achieving a successful motion detection method. The basic pipeline of these methods begins with loading the image sequence and converting the sequence to grayscale<sup>1</sup>. Then an image filter is applied to reduce the noise in the image followed by regularizing the pixel values limiting the range from 0-1 making it easier to process in the next step. Next, as enough frames are available, it applies a 1-D differential operator at each pixel in order to compute a temporal derivative. Then the absolute values of these derivatives are then reduced to either 0 or 1 on whether they exceed the threshold to create a black-white mask of the moving objects. Lastly, the mask is combined with the original frame to display results.

### A. Simple filter vs. 1D Derivative of a Gaussian

One variation attempted on this overview was to try a simple 0.5[-1,0,1] filter and a 1D derivative of a gaussian with a standard deviation  $t_0$ , defined by the user. Theoretically, the simple filter should be fast and quickly detect sharp motion, but may be sensitive to noise. The equation used for the simple filter is below:

$$\partial I / \partial t(x, y, t) \approx 0.5 \times [I(x, y, t + 1) - I(x, y, t - 1)] \quad (1)$$

The 1D derivative of a Gaussian uses convolution across the temporal dimension using frames neighboring to the selected one, with the  $\sigma_t$  parameter controlling the temporal smoothing. Advantages of this method should include reduced noise and smoother derivatives, with the downside of larger  $\sigma_t$  blurring rapid motion. The equation for this case:

$$G'(t; \sigma) = -(t/\sigma^2) \exp(-t^2/2\sigma^2) \quad (2)$$

### B. Spatial Smoothing Filter

Filters are kernels which are applied via a convolution over the image to reduce the noise in the images. For this there are several strategies, with the two used for this project being a box filter and a gaussian filter.

The box filter is the simplest where it's just an average of the pixels surrounding the center pixel. Equation 3 shows the matrices used as the box filters.

<sup>1</sup>The RGB values are averaged to get the value of the grayscale for this project.

$$B_{3 \times 3} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, B_{5 \times 5} = \frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \quad (3)$$

The other filter use is a Gaussian Filter. This is defined in the equations below with equation 4 being the 1D-Gaussian Filter and equation 5 being the 2D-Gaussian Filter.

$$\mathbf{g} = \alpha \begin{bmatrix} g_0 \\ g_1 \\ \vdots \\ g_{k-1} \end{bmatrix}, \quad g_i = \exp\left(-\frac{\left(i - \frac{k-1}{2}\right)^2}{2\sigma^2}\right) \quad (4)$$

In equation 4,  $\alpha$  is the regularization variable, so that the sum of all of the elements of  $g$  are 1. To get the 2d matrix so the filter can be applied to the image, equation 5.

$$\mathbf{G} = \mathbf{g} \mathbf{g}^\top \quad (5)$$

The matrix  $\mathbf{G}$  is use as the filter for the gaussian filters. For each of the filters of different sizes, a width of  $5\sigma$  is used. This is to ensure that all the filters capture 98% of the data, as at the extremes they are  $2.5\sigma$  from the mean.

To apply the filter, it is quite simple. To get the result just perform the calculation  $K * I$ , where  $K$  is the filter and  $I$  is the image to apply it on. To apply this in actuality Scipys convolve function [1] was used with the default padding setting of reflect. This reflects the numbers at the boarder, which prevents the boarders of the image from getting slightly darker, as there aren't a bunch of 0's reducing the value at the boarders.

### C. Adaptive Threshold Selection

The next step was testing varying threshold levels to achieve the mask. A fixed threshold doesn't work very well across scenes, as the level of complication in the scene can be dynamic. One solution to this is to estimate background noise levels automatically. Using Median Absolute Deviation (MAD), a method that is robust to outliers (moving objects in this case), with the assumption that the background follows a Gaussian distribution with zero mean, the equation below was used:

$$noise\_std = 1.4826 \times \text{median}(|derivative|) \quad (6)$$

$$threshold = n \times noise\_std \quad (7)$$

With the noise level, the threshold could then be found using a multiplier  $n$ , chosen to be  $n=3$ , which would capture 99.7% of the background noise. With this, pixels exceeding the threshold can be confidently classified as motion.

## III. EXPERIMENTS AND VALUES OF PARAMETERS USED

This was implemented using Python with the NumPy, OpenCV, and SciPy libraries [1]–[4]. The dataset included frames from a video of different scenes, that all consisted of 300-1100 frames. The frames were relatively low resolution JPEG files for fast processing. One video was set in a hallway with people passing through. Another was of an office room with a person walking by moving a chair into the scene. The third was of a room with a person moving a red chair into the room with a cluttered background.

### A. Experiment 1: Temporal Derivative Filters

The objective of this experiment was to compare simple vs. Gaussian derivative filters. A simple filter with parameters 0.5[-1, 0, 1] was used, and Gaussian filters of  $\sigma_t = 0.5, 1.0, 2.0$ . The method began with selecting the representative frame from the middle of the chosen sequence of frames. Then, it applied the simple and the three Gaussian filters separately, followed by computing the absolute temporal derivative.

### B. Experiment 2: Spatial Smoothing

The objective of this experiment was to find the ideal spacial filter to best aid the temporal derivative filter. Filter sizes of 3 by 3, 5 by 5, 7 by 7, 9 by 9, and 11 by 11 were tested using a  $\sigma$  of one fifth of the width of the filter (i.e. a 5 by 5 would be  $\sigma = 1$ ). These were used as it captures a majority of the distribution of the gaussian. After this the mask is made given the absolute value of the derivative exceeds the threshold set. The mask is then applied on the original image, giving the final result.

### C. Experiment 3: Adaptive Threshold

With the objective of designing an adaptive threshold selection method, parameters of  $n\_std = 1, 2, 3, 5, 10$ , and 20 were tested as they are multiples of noise standard deviation. Only the best spatial and temporal filter combination from the previous experiments was used, with only the threshold multiplier varying to assure control.

## IV. OBSERVATIONS

### A. Experiment 1: Temporal Derivative Filters

Figure 1, shows that the Gaussian Filters perform better than the simple temporal. Through the experiment, it was found that the simple filter was not able to deal with high levels of noise and led to high error though that path, producing noisier masks in uniform background regions. It demonstrated high sensitivity to rapid changes in pixel intensity while being able to detect sharp, fast-moving objects. As a whole, the Gaussian derivative filters were able to reduce noise through smoother temporal filtering and had better signal to noise ratio in derivative estimation. However, larger  $\sigma$  values blurred rapid motion, and were likely to miss fast-moving small objects. The  $\sigma = 0.5$  suffered from the same noise issues as the simple filter, and detected motion in many static pixels. The  $\sigma = 2.0$  resulted in over-smoothing effects and was not able to detect the sharp edges of the moving object. The  $\sigma = 1.0$  provided the

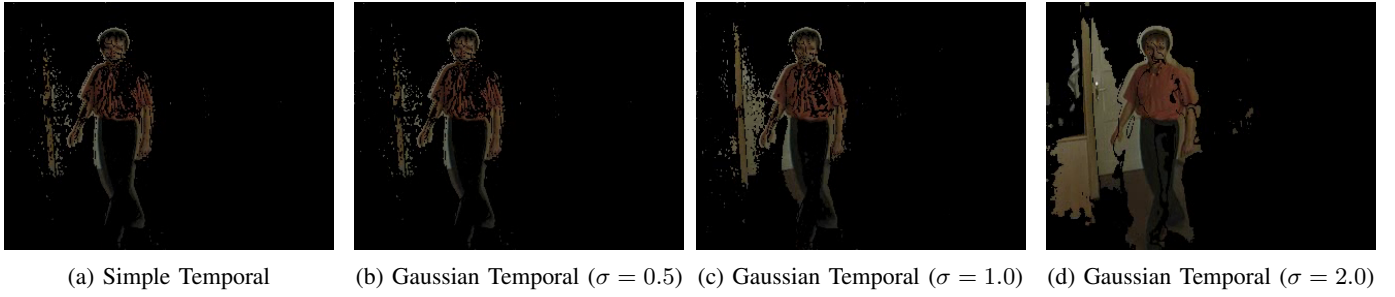


Fig. 1: Comparison of each of the Temporal Filters using a 3 by 3 Gaussian Image Filter from the Office Image Sequence. Images were taken from the 100 frame of the sequence. The automatic mask was used with a threshold set to  $3\sigma$ .

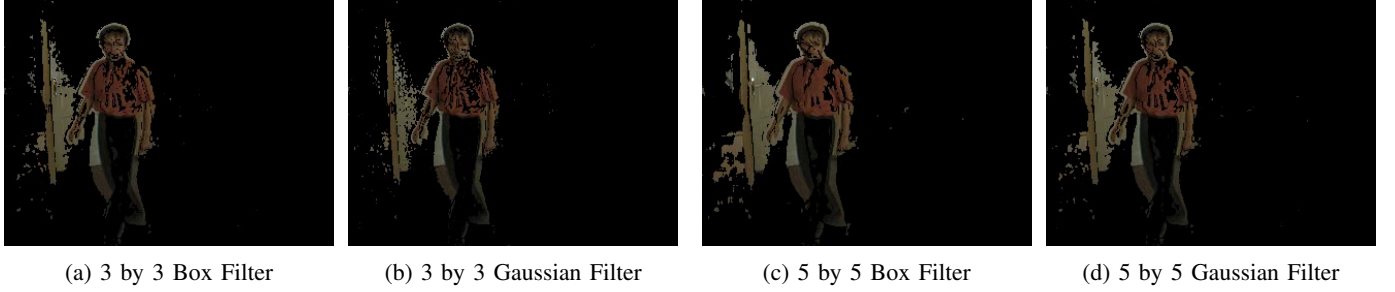


Fig. 2: Comparison of Box to Gaussian Image Filter using the Gaussian Temporal filter with a sigma of 1.0 using the Office Image Sequence. Images were taken from the 100 frame in the sequence. The automatic mask was used with a threshold set to  $3\sigma$ .

best balance with its ability to sharply detect the edges of the moving object, while not suffering too much from extraneous noise.

### B. Experiment 2: Spatial Smoothing

Figure 2, demonstrates how the gaussian filter can pick out much finer detail. While obviously the larger the filter size the less granular it is, as there are less fine details on the mask, the Gaussian was still able to be more fine in detail compared to the simple box filter. As can be seen comparing figures 2b & 2d to their corresponding filters in figures 2a & 2c, the mask for the gaussian captures much finer detailed lines, while the box filters tend to have larger clumps. Additionally as can be seen all of the filters captures part of the wall in addition to the person moving. As can be seen the Gaussian Filter does a better job in both sizes of removing the wall compared to the box filters.

Figure 3 shows how the filter size has an effect on resultant mask. As can be seen, the larger the size, the smoother the boundary is. This is most clear when comparing figures 3b and 3f. As can be seen this is a much smoother boundary in the mask with the 11 by 11 compared to the 3 by 3 Gaussian filters. In contrast the 3 by 3 gave a more fine grained mask compared to the 11 by 11 filter. The 5 by 5 gaussian filter performs the best of them all, as it maintains the detail from the 3 by 3 gaussian, but also has a great reduction in noise greatly compared to the 3 by 3. Something to note across all of the images is that part of the person is masked though they are moving. This is the result of while they may be moving,

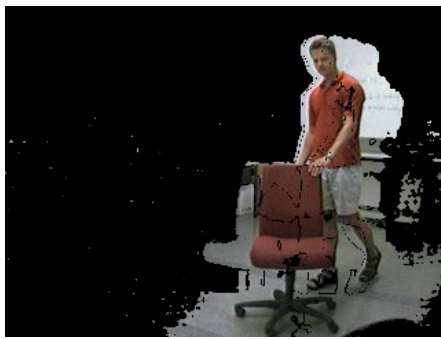
their shirt is all the same color which means there isn't any large changes in the color of that pixel in near by frames. Additionally since this is being done in grayscale, as long as the intensity of the pixel stays the same, then a change will not be detected.

### C. Experiment 3: Adaptive Threshold

Figure 4 demonstrated that even at extremely high set thresholds, the motion was able to be captured. The EnterExitCrossingPaths2cor dataset was used to test the best automatic threshold used as it contains a lot of noise and motion, which makes it perfect for testing removing noise. Thresholds of 1, 2, 3, 5, 10, and 20 standard deviations were tested. While 20 standard deviations is extreme, containing  $1 - 5.507 \times 10^{-89}$  of the noise, it was still able to show the motion as seen in figure 4f. While 20 standard deviations does end up masking significantly more of the objects in motion, the ability to still be able to get any data above the threshold demonstrates that a higher adaptive threshold is better as it reduces the noise significantly while still being capable of detecting motion. When looking at all the images, a standard deviation of 5 seems to be the most effective, as it removes almost all the noise that the higher standard deviations removed, while also preventing the least masking over the object in motion.

## V. CONCLUSIONS

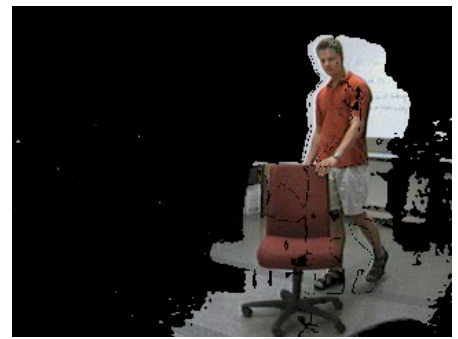
It was found while the simpler filters for both the temporal and image worked, there were not as effective as the gaussian versions of the filter. For the temporal filter a  $\sigma$  of 1.0 was



(a) 3 by 3 Box Filter



(b) 3 by 3 Gaussian Filter



(c) 5 by 5 Gaussian Filter



(d) 7 by 7 Gaussian Filter

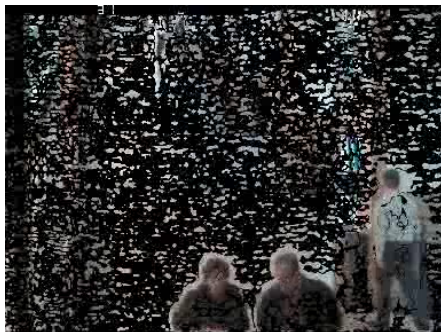


(e) 9 by 9 Gaussian Filter

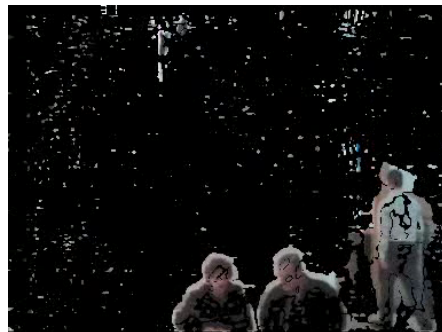


(f) 11 by 11 Gaussian Filter

Fig. 3: Comparison of Gaussian Image Filter Sizes using the Gaussian Temporal filter with a sigma of 1.0 using the Office Image Sequence. These pictures were taking in frame 55. The automatic mask was used with a threshold set to  $3\sigma$ .



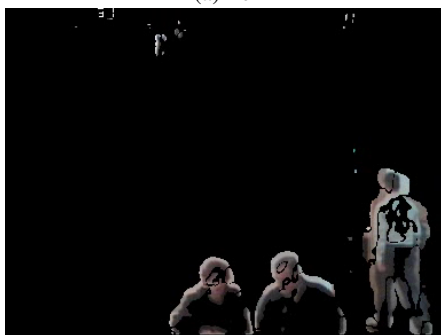
(a)  $1\sigma$



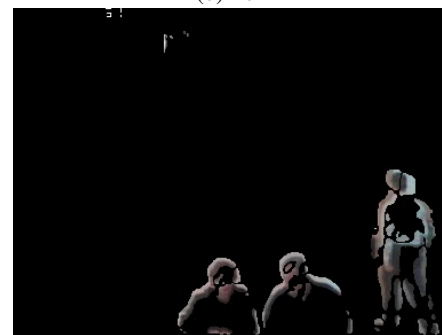
(b)  $2\sigma$



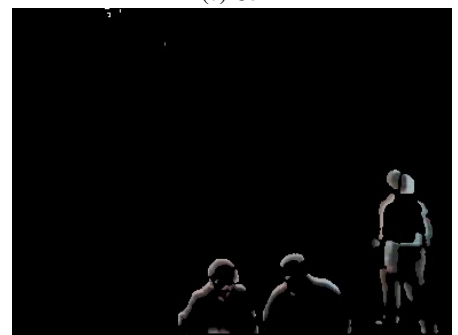
(c)  $3\sigma$



(d)  $5\sigma$



(e)  $10\sigma$



(f)  $20\sigma$

Fig. 4: Comparison of Different Adaptive Threshold Standard Deviations from the Mean. A Gaussian 5 by 5 image filter was used with a Gaussian Temporal filter with a sigma of 1.0 using the EnterExitCrossingPaths2cor Images. These pictures were taking in frame 45.

found to be the most effect. For the image filter a  $\sigma$  of 1.0 for a 5 by 5 filter was the most effective. As for the automatic threshold, a standard deviation of 5 was found to be the best threshold as it removed almost all of the noise while minimizing masking the objects in motion.

It was found that regardless of the settings, the objects in motion always had some amount of mask on the object. This was likely due to two major reasons: first objects moving having similar colors on the object, which results in small changes in intensity on the object; second the image being converted into a grayscale, which reduces the information it can use for detecting motion. Reducing it to grayscale means only one channel, the intensity, is being used rather than multiple channels of color as a means to understand whether a pixel changes or not. If the color shifts, but the average of the channels stay the same, this will result in the same value for the grayscale, which results in measured derivative of zero. A future version could perform this algorithm on all of the color channels and then at the end combine the derivatives post processing to better detect motion. However, this could also potentially result in more noise being detected as motion.

#### REFERENCES

- [1] "SciPy." [Online]. Available: <https://scipy.org/>
- [2] "Welcome to Python.org," Feb. 2026. [Online]. Available: <https://www.python.org/>
- [3] "NumPy." [Online]. Available: <https://numpy.org/>
- [4] "OpenCV - Open Computer Vision Library." [Online]. Available: <https://opencv.org/>